Relay: AI Workflow Orchestrator

Case Study Engineering Approach & Technical Specification | AI-First Capstone

1. Executive Summary & Problem Understanding

Enterprise operations rely heavily on automated workflows (such as customer support triage, order fulfillment, and expense approvals). Modern workflow engines like Temporal, n8n, or Zapier solve deterministic execution, but introducing AI judgment directly into workflow nodes presents four critical engineering vulnerabilities:

- **Schema Drift & Malformed Output:** LLM outputs are inherently stochastic and prone to schema violations or hallucinations, breaking downstream deterministic branching.
- **Adversarial Prompt Injections:** Untrusted webhook/ticket payloads contain adversarial prompt injections attempting to bypass approval policies (e.g., forcing auto-refunds).
- **Side-Effect Duplication on Worker Crashes:** Replaying a crashed workflow from step 1 triggers duplicate external side effects (duplicate billing, duplicate refund calls, redundant Slack alerts).
- **Runaway Cyclic Loops:** Dynamic polling loops and unconstrained cycles can run indefinitely, exhausting cloud infrastructure and API budgets.

2. Proposed Solution Architecture

Relay implements a resilient, state-machine-driven orchestrator designed with asynchronous queue processing and defense-in-depth isolation:

- **Workflow Lifecycle & Snapshotting:** Workflows transition through Draft -> Published states with publish-time DAG validation. At trigger time, an immutable snapshot of the workflow graph is recorded so runtime edits never corrupt active executions.
- **Durable Queue & Step Checkpointing:** Triggers immediately enqueue runs. An asynchronous worker executes nodes sequentially, writing checkpoints to SQLite/PostgreSQL after every step. Upon worker restarts, in-flight runs resume strictly from the last completed checkpoint.
- **Exactly-Once Semantics & Idempotency:** All mutating external calls (notify, order actions, HTTP mutations) send deterministic idempotency keys ({run_id}:{node_id}:{attempt}) to guarantee exactly-once execution at the service ledger.
- **Schema-Enforced AI Decisions:** AI decision nodes require strict JSON Schema validation. If validation fails, the engine retries once with error feedback before terminating cleanly.
- **Engine-Enforced Human Approval Gates:** Sensitive nodes (requires_approval: true) are structurally unreachable without an explicit database approval record, rendering prompt injections completely ineffective.
- **Runaway Bounds & Trace Observability:** Hard limits.max_steps cap (default 20 steps) terminates runaway loops (e.g. wf_runaway), preserving full audit trails in the run trace console.
- **(Bonus) Natural Language Workflow Compiler:** An AI-powered compiler (/workflows/compile) that transforms natural language text into valid, schema-compliant workflow DAGs while rejecting impossible requests.

3. Implementation Roadmap & Milestones

Milestone	Core Deliverables	Validation / Exit Criteria
M1: Core Platform & State Machine	Ingest node catalog & seed workflows; build Draft/Publish CRUD APIs; snapshot frozen definitions.	Publish-time graph validation green; immutable trigger snapshots verified.
M2: Async Engine & Deterministic Nodes	Database queue & async worker loop; implement condition, http_request, delay, and notify nodes with template resolution.	Non-blocking trigger responses; deterministic node branching verified.
M3: Durability, Idempotency & Approvals	Per-step state checkpointing; deterministic idempotency keys against mock world; human approval gate node.	Crash-resume test passes with zero duplicate ledger side effects (duplication_check.py).
M4: AI Robustness & Security Guardrails	AI node adapter with JSON Schema enforcement; single auto-retry on schema error; injection-safe handling.	pay_inject_001 blocked from bypassing approval gates; schema validation prevents broken branches.
M5: Safety Caps, Operator UI & Verification	Enforce limits.max_steps cap; interactive Web Console for traces/approvals; end-to-end smoke testing.	wf_runaway stopped at step cap; scripts/smoke_test.py passes 100%.
M6: (Bonus) Natural Language Compiler	NL workflow compiler endpoint with structural constraint checks and error explanations.	Eval benchmark data/nl_eval.jsonl tested with high synthesis accuracy.

4. Verification Strategy & Acceptance Criteria

- **Contract Testing:** scripts/smoke_test.py succeeds with 100% pass rate across core lifecycle and API endpoints.
- **Crash-Recovery & Idempotency:** Live worker termination during wf_slow_fulfillment resumes correctly; scripts/duplication_check.py verifies zero duplicate ledger entries.
- **Security Verification:** Adversarial payloads (pay_inject_001/002) fail to bypass approval gates due to engine-enforced isolation.
- **Resource Bound Assurance:** wf_runaway is cleanly aborted when exceeding maximum allowable steps with status 'failed'.